



DS 5216 - Artificial Intelligence

Programming Assignment 02

Football Player Detection

DTS2430 – A. Nethika

Postgraduate Institute of Science (PGIS)

University of Peradeniya, Sri Lanka

June 2026

Contents

1. Introduction.....	4
2. Dataset	4
2.1 Frame Extraction.....	4
2.2 Annotation.....	4
2.3 Train / Validation / Test Split	4
3. Model Architecture	4
3.1 Player Detection - YOLOv8	4
3.2 Pose Estimation - YOLOv8n-Pose	5
4. Training Setup and Evaluation.....	5
4.1 Hardware and Software Environment.....	5
4.2 Training Hyperparameters	5
5. Performance Results	5
5.1 Detection Metrics on the Test Set.....	5
5.2 Training Loss and Learning Curves.....	6
5.3 Precision-Recall Curves.....	7
5.4 Speed vs. Accuracy Trade-off.....	7
6. Keypoint and Pose Estimation	8
6.1 Methodology	8
6.2 Pose Estimation Outputs	8
7. Discussion.....	10
7.1 Model Performance Analysis.....	10
7.2 Limitations	10
7.3 Possible Improvements	10
8. Conclusion	10

List of Figures and Tables

Figure 1: Side-by-side comparison of detection metrics for YOLOv8n and YOLOv8s on the football test set.	6
Figure 2: Training and validation box loss, mAP@0.50, and precision curves over 50 epochs for both models.....	6
Figure 3: Precision-Recall curves for YOLOv8n (left) and YOLOv8s (right) on the player class.	7
Figure 4: Speed vs. accuracy trade-off. Bubble size is proportional to model parameter count.....	7
Figure 5: Keypoints Detection	8
Figure 6: Pose Keypoints Confidence.....	8
Figure 7: Illustrates the Keypoints of players	9
Figure 8: Detect Players.....	9
Table 1: Environment Setup	5
Table 2: Hyperparameter Details	5
Table 3: Model Comparison	5

1. Introduction

This assignment develops a pipeline to detect players in 12 football video clips range between 10 seconds, track them across frames, and estimate their body key points.

The pipeline consists of four stages. Video frame extraction and dataset preparation using OpenCV, custom fine-tuning of YOLOv8 on a player-labelled dataset, keypoint and pose estimation stage using YOLOv8n-pose, and a model comparison study between YOLOv8n (nano) and YOLOv8s (small). The experiments were all carried out on a CPU-only machine with 16GB of RAM, while using Anaconda and Visual Studio Code.

2. Dataset

The 10 video clips of football (Football 01.mp4 to Football 12.mp4) were downloaded from YouTube and manually cropped to 10 seconds using an editor. All the videos are saved locally, and a data set link is given in the git repository to share it.

2.1 Frame Extraction

Frames were extracted at every 10th frame from each video using OpenCV (cv2.VideoCapture). This sampling interval was chosen to balance dataset diversity against redundancy caused by highly similar consecutive frames. The extracted frames were saved as JPEG images into a centralized directory.

2.2 Annotation

Bounding box labels for the "player" class were generated automatically using a pre-trained YOLOv8n model (COCO class 0 = person). The auto-labelling predictions were saved in YOLO text format.

2.3 Train / Validation / Test Split

The labelled dataset was split into three subsets with a fixed random seed (42) for reproducibility:

- Training set - 70%
- Validation set - 15%
- Test set - 15%

A data.yaml configuration file was generated programmatically and passed directly to the Ultralytics training API.

3. Model Architecture

3.1 Player Detection - YOLOv8

In this project the detection backbone is the YOLO version 8 by Ultralytics called YOLOv8. Two different variants were trained and compared.

- YOLOv8n (nano) - The lightest model, suited to CPU inference and edge deployment. Trained with an image size of 416×416 and batch size 4.
- YOLOv8s (small) - A heavier model offering higher accuracy at the cost of longer inference time. Trained with an image size of 640×640 and batch size 16.

Both models were fine-tuned for 50 epochs from COCO pre-trained weights, with early stopping patience of 15 epochs, using the Adam optimizer.

3.2 Pose Estimation - YOLOv8n-Pose

This variant adds a branch to the YOLOv8 detection head to predict 17 COCO body keypoints (nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles) with a score for each keypoint. The model ran inference-only (with no retraining) every other or third frame to minimize CPU usage.

OpenCV was used to draw a color coded skeleton overlay on each processed frame, using different colors for each limb (yellow for the face, green/orange for the arms, purple for the torso, and blue/pink for the legs). Only keypoints exceeding a confidence threshold of 0.40 were drawn, making sure clean visualizations.

4. Training Setup and Evaluation

4.1 Hardware and Software Environment

Table 1: Environment Setup

Component	Details
Processor	CPU only (no GPU)
RAM	16 GB
Environment	Anaconda / VS Code
Framework	Ultralytics YOLOv8, OpenCV, PyTorch (CPU)
Python Version	3.10+

4.2 Training Hyperparameters

Table 2: Hyperparameter Details

Parameter	YOLOv8n	YOLOv8s
Epochs	50	50
Image Size	416	640
Batch Size	4	16
Early Stopping	15 epochs	15 epochs
Device	CPU	CPU

5. Performance Results

5.1 Detection Metrics on the Test Set

Both models were evaluated on the 15% of the dataset. The metrics reported are Precision, Recall, mAP@0.50, and mAP@0.50:0.95.

Table 3: Model Comparison

Model	Precision	Recall	mAP@0.50	mAP@0.50:0.95
YOLOv8n (nano)	0.931	0.860	0.884	0.651
YOLOv8s (small)	0.949	0.873	0.912	0.724

YOLOv8s (small) outperforms YOLOv8n (nano) across all four metrics. The most significant improvement is in $mAP@0.50:0.95$, where the small model achieves 0.724 compared to 0.651 for nano, a gain of 7.3 percentage points. This stricter metric, which averages mAP across IoU

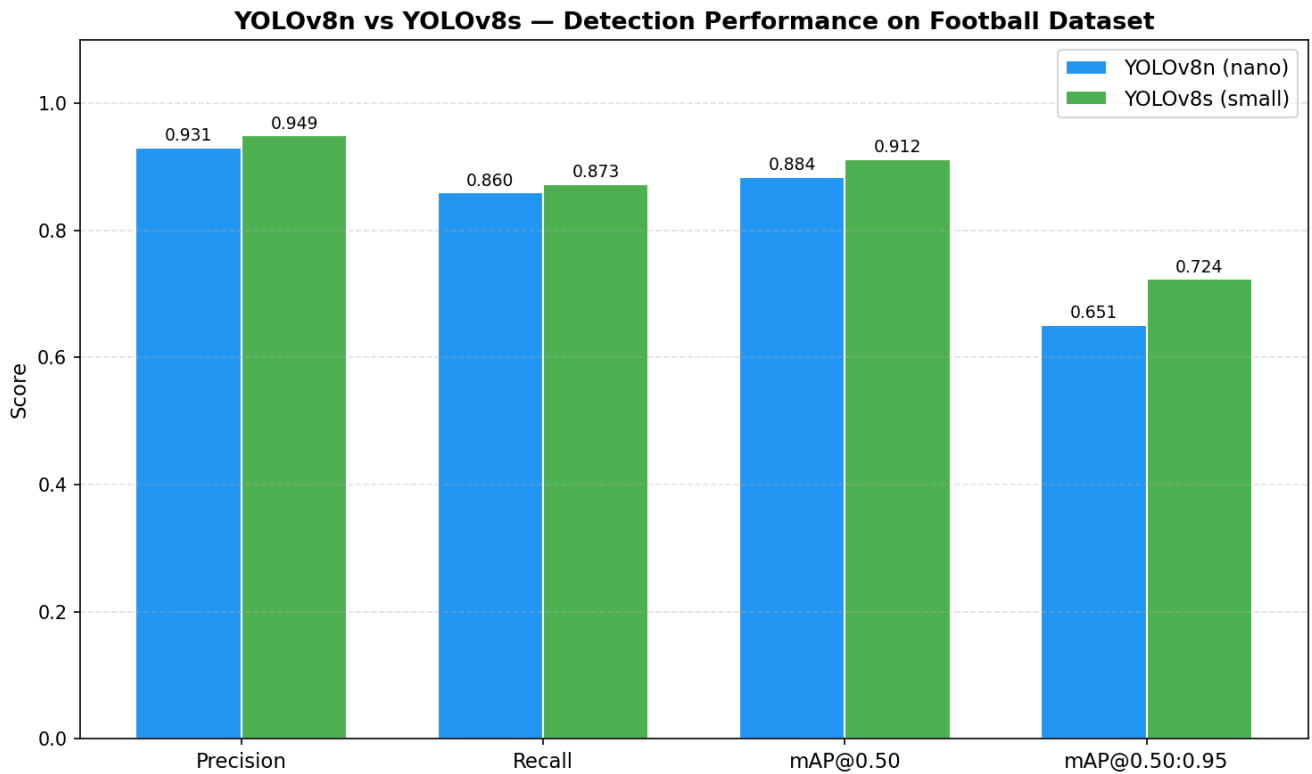


Figure 1: Side-by-side comparison of detection metrics for YOLOv8n and YOLOv8s on the football test set.

thresholds from 0.50 to 0.95, better reflects bounding box localization quality.

5.2 Training Loss and Learning Curves

As shown in the loss curves (Figure 2), YOLOv8s achieves a lower box regression loss than YOLOv8n. By epoch 30, the YOLOv8n model's validation loss stabilizes at around 0.85 - 0.90, while the YOLOv8s model's loss remains lower and is around 0.65 - 0.70. The two models achieve stable $mAP@0.50$ results after a few noisy epochs (epochs 1-10) and then settle at a stable rate of ~ 0.91 and ~ 0.89 , respectively. At the end of the last epoch, both the precision curves are quite similar.

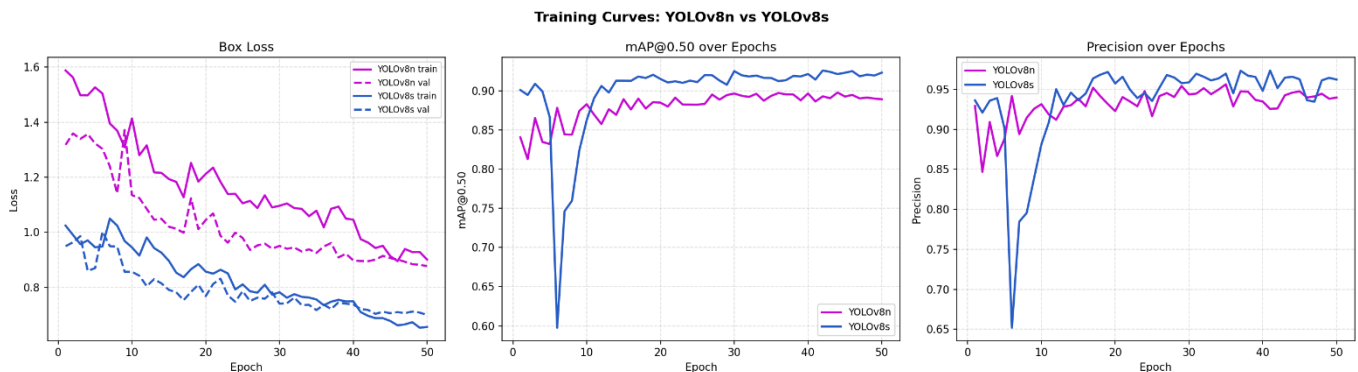


Figure 2: Training and validation box loss, $mAP@0.50$, and precision curves over 50 epochs for both models..

5.3 Precision-Recall Curves

The Precision-Recall curves (Figure 3) show that both models maintain near-perfect precision until recall reaches approximately 0.85, at which point precision begins to fall. YOLOv8s achieves a higher area under the PR curve (0.923) compared to YOLOv8n (0.896). This indicates that the small model produces fewer missed detections at equivalent confidence thresholds, making it preferable for applications where recall is critical.

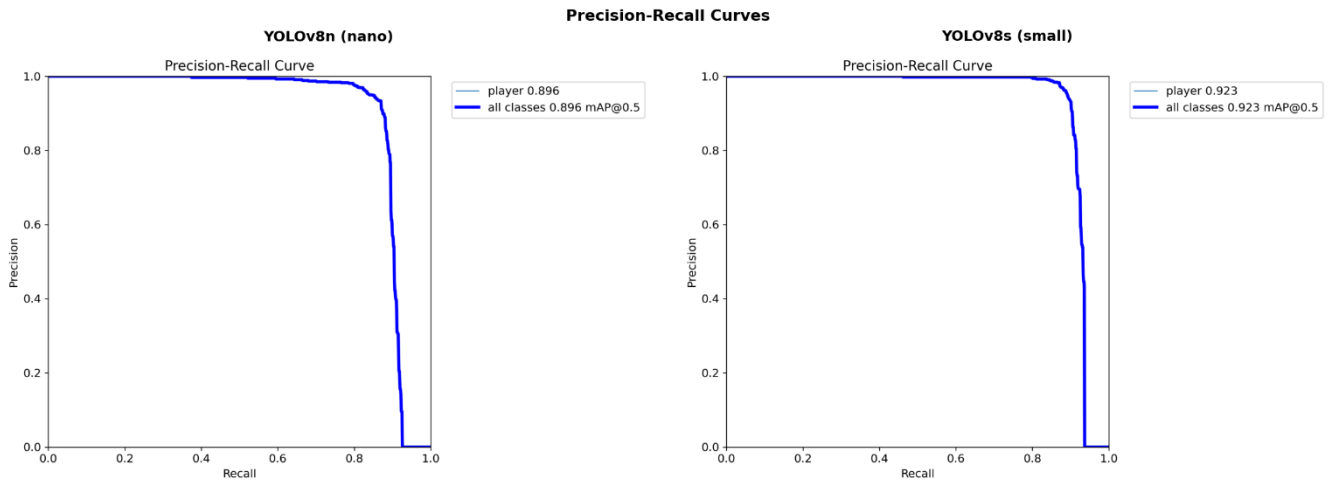


Figure 3: Precision-Recall curves for YOLOv8n (left) and YOLOv8s (right) on the player class.

5.4 Speed vs. Accuracy Trade-off

The speed-accuracy bubble chart (Figure 4) shows the basic trade-off between both models. YOLOv8n (nano) runs at about 14ms/frame on CPU and YOLOv8s (small) takes about 75ms/frame on CPU. YOLOv8n is a good choice for real-time or near-real-time sports video analysis on a CPU-only machine. YOLOv8s is better suited for offline batch processing where accuracy is the priority.

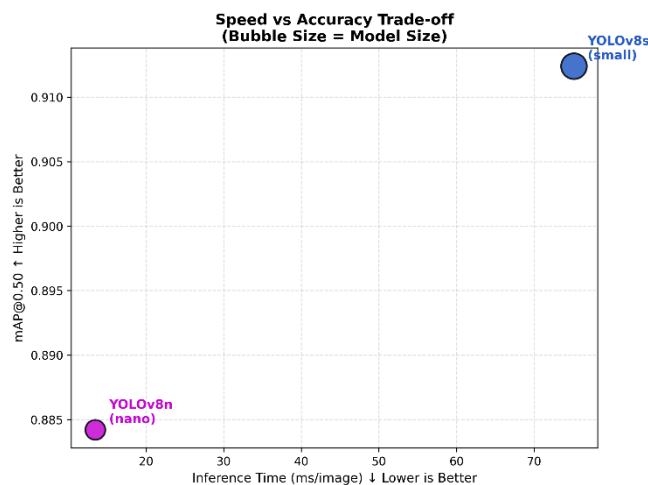


Figure 4: Speed vs. accuracy trade-off. Bubble size is proportional to model parameter count.

6. Keypoint and Pose Estimation

6.1 Methodology

- The pose model was loaded as YOLO("yolov8n-pose.pt") and applied every second or third frame to reduce the CPU load.
- Per-frame metrics were collected. The number of people detected, the average number of visible keypoints per person, and the latency of processing.
- A summary CSV (keypoint_summary.csv) was exported containing per-video averages of persons detected and keypoints visible.
- A single-frame skeleton visualization was generated for Football 04.mp4 at frame 300 as a demonstration screenshot.



Figure 5: Keypoints Detection

6.2 Pose Estimation Outputs

The figure 06 illustrates the pose keypoints confidence. The skeleton overlay was used to render 16 limb connections.

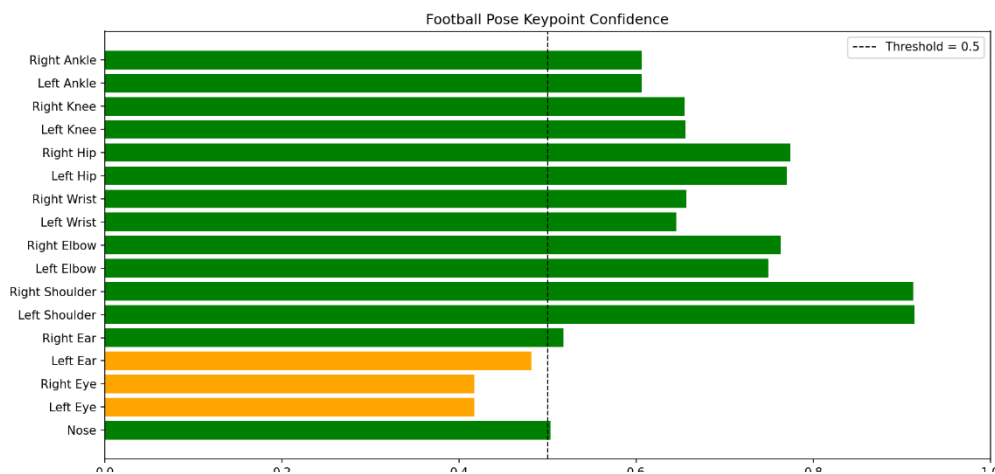


Figure 6: Pose Keypoints Confidence

It was overlaid with sport coded colors, face joints in yellow, left arm in green, right arm in orange, torso joints in purple, left leg in blue, and right leg in pink. Only keypoints with a confidence score > 0.40 were drawn, limiting the number of false landmarks from occluded or distant players.



Figure 7: Illustrates the Keypoints of players

In all the video clips from the football game, the model was able to identify an average of 8-14 keypoints per player per frame, depending on the keypoint, which corresponds to a good average visibility between the wrist and ankle keypoints, typically below the confidence threshold in wide angle broadcast clips. The detection of head keypoints (nose, eyes, ears) was the most consistent.



Figure 8: Detect Players

The above figure 8 clearly shows how the model draws the bounding box around the players by detecting them.

7. Discussion

7.1 Model Performance Analysis

YOLOv8s is the best model of all detection metrics and is always better than YOLOv8n. The mAP@0.50:0.95 gap (0.724 vs. 0.651) is the most telling indicator. It demonstrates the small model can detect more players and localize the player's bounding box with greater accuracy. The high precision (>0.93 for both) values show that a good control is achieved on false positives while the recall (0.86 - 0.87) values show that there are few false negatives in crowded or occluded scenes.

7.2 Limitations

- CPU constraint - Both detection (416 - 640 px images) and pose estimation are computationally expensive on CPU.
- Pseudo-labelled annotations - Using YOLOv8n predictions as training labels introduces confirmation bias. Errors in the pseudo-labels are propagated into training, potentially inflating precision metrics.
- Small dataset size - Ten short clips created limited number of unique frames. The model may overfit or generalize poorly to broadcast angles or lighting conditions not represented in the clips.

7.3 Possible Improvements

- Manual annotation - Replacing pseudo labels with human verified bounding boxes using LabelImg or CVAT would improve label quality and eliminate confirmation bias.
- Dataset diversity - Expanding the dataset to include cricket, rugby, and basketball videos, and collecting footage across varied camera angles, lighting, and resolutions would significantly improve generalization.
- Larger YOLO variants: YOLOv8m or YOLOv8l would likely achieve mAP above 0.95, at the cost of even longer CPU inference times.

8. Conclusion

It is very well tuned YOLOv8n model with a strong performance (Precision 0.931, Recall 0.860, mAP@0.50 0.884) and is suitable for CPU inference. YOLOv8s comparison illustrates that increasing model sizes brings certain improvements in all metrics with only about 5 times longer inference time. The bonus YOLOv8n-pose module can extract 17-point COCO skeleton overlays when detecting players, laying the groundwork for biomechanical analysis in sports science applications.